Pragati Engineering College
(Autonomous)

PROJECT :

Smart Personal Food Safety and Healthcare Assistant

Team leader : M.Abhishek (25A31A0543)

Team members : Ch.Sandhya (25A31A0504)

P.Sanjana(25A31A0524)

P.Pavan(25A31A0557)

Smart Personal Food Safety and Healthcare Assistant

Code for the project in C++ :

```
#include <iostream>
#include <vector>
#include <string>
#include <fstream>
#include <sstream>
#include <iomanip>
#include <ctime>
#include <algorithm>

using namespace std;

struct Date {
    int day, month, year;
};

struct FoodItem {
    int id;
    string name;
    string category;
    int quantity;
    Date purchaseDate;
    Date expiryDate;
    string storageType;
    float protein;
    string vitamins;
    float sugar;
    float sodium;
```

```
    bool badSmell;
    bool colorChanged;
    bool leakage;
    bool fungus;
    int leftoverDays;
};

struct Patient {
    int id;
    string name;
    int age;
    string disease;
    string allergy;
};

struct HospitalContact {
    int id;
    string name;
    string phone;
    string location;
};

class FileManager {
public:
    static void saveFoods(const vector<FoodItem>& foods) {
        ofstream file("foods.txt");
        for (const auto& f : foods) {
            file << f.id << "|"
                << f.name << "|"
                << f.category << "|"
                << f.quantity << "|"
                << f.purchaseDate.day << " " << f.purchaseDate.month << " " <<
f.purchaseDate.year << "|"
```

```
            < f.expiryDate.day << " " << f.expiryDate.month << " " <<
f.expiryDate.year << "|"
            << f.storageType << "|"
            << f.protein << "|"
            << f.vitamins << "|"
            << f.sugar << "|"
            << f.sodium << "|"
            << f.calories << "|"
            << f.badSmell << "|"
            << f.colorChanged << "|"
            << f.leakage << "|"
            << f.fungus << "|"
            << f.leftoverDays << "\n";
        }
        file.close();
    }

    static void loadFoods(vector<FoodItem>& foods) {
        foods.clear();
        ifstream file("foods.txt");
        string line;
        while (getline(file, line)) {
            stringstream ss(line);
            FoodItem f;
            string temp;

            getline(ss, temp, '|'); f.id = stoi(temp);
            getline(ss, f.name, '|');
            getline(ss, f.category, '|');
            getline(ss, temp, '|'); f.quantity = stoi(temp);

            getline(ss, temp, '|');
            stringstream pd(temp);
```

```
            pd >> f.purchaseDate.day >> f.purchaseDate.month >>
f.purchaseDate.year;

            getline(ss, temp, '|');
            stringstream ed(temp);
            ed >> f.expiryDate.day >> f.expiryDate.month >> f.expiryDate.year;

            getline(ss, f.storageType, '|');
            getline(ss, temp, '|'); f.protein = stof(temp);
            getline(ss, f.vitamins, '|');
            getline(ss, temp, '|'); f.sugar = stof(temp);
            getline(ss, temp, '|'); f.sodium = stof(temp);
            getline(ss, temp, '|'); f.calories = stof(temp);
            getline(ss, temp, '|'); f.badSmell = stoi(temp);
            getline(ss, temp, '|'); f.colorChanged = stoi(temp);
            getline(ss, temp, '|'); f.leakage = stoi(temp);
            getline(ss, temp, '|'); f.fungus = stoi(temp);
            getline(ss, temp, '|'); f.leftoverDays = stoi(temp);

            foods.push_back(f);
        }
        file.close();
    }

    static void savePatients(const vector<Patient>& patients) {
        ofstream file("patients.txt");
        for (const auto& p : patients) {
            file << p.id << "|"
                 << p.name << "|"
                 << p.age << "|"
                 << p.disease << "|"
                 << p.allergy << "\n";
        }
        file.close();
```

```
}

static void loadPatients(vector<Patient>& patients) {
    patients.clear();
    ifstream file("patients.txt");
    string line;
    while (getline(file, line)) {
        stringstream ss(line);
        Patient p;
        string temp;

        getline(ss, temp, '|'); p.id = stoi(temp);
        getline(ss, p.name, '|');
        getline(ss, temp, '|'); p.age = stoi(temp);
        getline(ss, p.disease, '|');
        getline(ss, p.allergy, '|');

        patients.push_back(p);
    }
    file.close();
}

static void saveHospitals(const vector<HospitalContact>& hospitals) {
    ofstream file("hospitals.txt");
    for (const auto& h : hospitals) {
        file << h.id << "|"
             << h.name << "|"
             << h.phone << "|"
             << h.location << "\n";
    }
    file.close();
}

static void loadHospitals(vector<HospitalContact>& hospitals) {
```

```
        hospitals.clear();
        ifstream file("hospitals.txt");
        string line;
        while (getline(file, line)) {
            stringstream ss(line);
            HospitalContact h;
            string temp;

            getline(ss, temp, '|'); h.id = stoi(temp);
            getline(ss, h.name, '|');
            getline(ss, h.phone, '|');
            getline(ss, h.location, '|');

            hospitals.push_back(h);
        }
        file.close();
    }
};

class SafetyEngine {
public:
    static Date getCurrentDate() {
        time_t t = time(0);
        tm* now = localtime(&t);
        return {now->tm_mday, now->tm_mon + 1, now->tm_year + 1900};
    }

    static int dateToNumber(Date d) {
        return d.year * 10000 + d.month * 100 + d.day;
    }

    static bool isExpired(Date expiry) {
        Date today = getCurrentDate();
        return dateToNumber(today) > dateToNumber(expiry);
```

```
    }

    static string checkFoodSafety(const FoodItem& f) {
        if (isExpired(f.expiryDate))
            return "UNSAFE: Food is expired.";

        if (f.leftoverDays > 4)
            return "UNSAFE: Leftover stored for more than 4 days.";

        if (f.badSmell || f.colorChanged || f.leakage || f.fungus)
            return "UNSAFE: Spoilage signs detected.";

        return "SAFE TO EAT";
    }
};

class DietEngine {
public:
    static string toLowerCase(string s) {
        transform(s.begin(), s.end(), s.begin(), ::tolower);
        return s;
    }

    static string canPatientEat(const Patient& p, const FoodItem& f) {
        string disease = toLowerCase(p.disease);
        string allergy = toLowerCase(p.allergy);
        string foodName = toLowerCase(f.name);
        string vitamins = toLowerCase(f.vitamins);

        if (!allergy.empty() && foodName.find(allergy) != string::npos)
            return "NOT ALLOWED: Allergy detected.";

        if (disease == "diabetes" && f.sugar > 10)
            return "AVOID: High sugar food for diabetic patient.";
```

```
        if ((disease == "bp" || disease == "hypertension") && f.sodium > 200)
            return "AVOID: High sodium food for BP patient.";

        if (disease == "kidney" && f.sodium > 150)
            return "AVOID: Not suitable for kidney patient.";

        return "ALLOWED: Patient can eat this food.";
    }
};

class GroceryCheck {
public:
    static int qualityScore(const FoodItem& f) {
        int score = 100;

        if (SafetyEngine::isExpired(f.expiryDate)) score -= 40;
        if (f.badSmell) score -= 20;
        if (f.colorChanged) score -= 15;
        if (f.leakage) score -= 15;
        if (f.fungus) score -= 20;
        if (f.leftoverDays > 4) score -= 20;

        if (score < 0) score = 0;
        return score;
    }

    static string qualityStatus(int score) {
        if (score >= 80) return "Excellent";
        if (score >= 60) return "Good";
        if (score >= 40) return "Average";
        return "Poor";
    }
};
```

```
class App {
private:
    vector<FoodItem> foods;
    vector<Patient> patients;
    vector<HospitalContact> hospitals;

public:
    void addFood() {
        FoodItem f;
        cout << "\nEnter Food ID: ";
        cin >> f.id;
        cin.ignore();

        cout << "Enter Food Name: ";
        getline(cin, f.name);

        cout << "Enter Category: ";
        getline(cin, f.category);

        cout << "Enter Quantity: ";
        cin >> f.quantity;

        cout << "Enter Purchase Date (dd mm yyyy): ";
        cin >> f.purchaseDate.day >> f.purchaseDate.month >> f.purchaseDate.year;

        cout << "Enter Expiry Date (dd mm yyyy): ";
        cin >> f.expiryDate.day >> f.expiryDate.month >> f.expiryDate.year;
        cin.ignore();

        cout << "Enter Storage Type (Room/Refrigerator/Freezer): ";
        getline(cin, f.storageType);

        cout << "Enter Protein (g): ";
        cin >> f.protein;
```

```
    cin.ignore();

    cout << "Enter Vitamins (example: A,B,C): ";
    getline(cin, f.vitamins);

    cout << "Enter Sugar (g): ";
    cin >> f.sugar;

    cout << "Enter Sodium (mg): ";
    cin >> f.sodium;

    cout << "Enter Calories: ";
    cin >> f.calories;

    cout << "Bad smell? (1 yes / 0 no): ";
    cin >> f.badSmell;

    cout << "Color changed? (1 yes / 0 no): ";
    cin >> f.colorChanged;

    cout << "Leakage? (1 yes / 0 no): ";
    cin >> f.leakage;

    cout << "Fungus visible? (1 yes / 0 no): ";
    cin >> f.fungus;

    cout << "Leftover stored days: ";
    cin >> f.leftoverDays;

    foods.push_back(f);
    cout << "Food item added successfully.\n";
}

void viewFoods() {
```

```
        if (foods.empty()) {
            cout << "\nNo food records found.\n";
            return;
        }

        for (const auto& f : foods) {
            cout << "\n----------------------------\n";
            cout << "ID: " << f.id << "\n";
            cout << "Name: " << f.name << "\n";
            cout << "Category: " << f.category << "\n";
            cout << "Quantity: " << f.quantity << "\n";
            cout << "Purchase Date: " << f.purchaseDate.day << "/" <<
f.purchaseDate.month << "/" << f.purchaseDate.year << "\n";
            cout << "Expiry Date: " << f.expiryDate.day << "/" << f.expiryDate.month
<< "/" << f.expiryDate.year << "\n";
            cout << "Storage: " << f.storageType << "\n";
            cout << "Protein: " << f.protein << " g\n";
            cout << "Vitamins: " << f.vitamins << "\n";
            cout << "Sugar: " << f.sugar << " g\n";
            cout << "Sodium: " << f.sodium << " mg\n";
            cout << "Calories: " << f.calories << "\n";
        }
    }

    FoodItem* findFoodById(int id) {
        for (auto& f : foods) {
            if (f.id == id) return &f;
        }
        return nullptr;
    }

    Patient* findPatientById(int id) {
        for (auto& p : patients) {
            if (p.id == id) return &p;
```

```
    }
    return nullptr;
}

void checkExpiryFoods() {
    if (foods.empty()) {
        cout << "\nNo food items available.\n";
        return;
    }

    for (const auto& f : foods) {
        cout << "\nFood: " << f.name << " -> ";
        if (SafetyEngine::isExpired(f.expiryDate))
            cout << "EXPIRED\n";
        else
            cout << "NOT EXPIRED\n";
    }
}

void checkFoodSafety() {
    int id;
    cout << "\nEnter Food ID to check safety: ";
    cin >> id;

    FoodItem* f = findFoodById(id);
    if (!f) {
        cout << "Food item not found.\n";
        return;
    }

    cout << SafetyEngine::checkFoodSafety(*f) << "\n";
}

void showNutrition() {
```

```
    int id;
    cout << "\nEnter Food ID to view nutrition: ";
    cin >> id;

    FoodItem* f = findFoodById(id);
    if (!f) {
        cout << "Food item not found.\n";
        return;
    }

    cout << "\nNutrition Details for " << f->name << ":\n";
    cout << "Protein: " << f->protein << " g\n";
    cout << "Vitamins: " << f->vitamins << "\n";
    cout << "Sugar: " << f->sugar << " g\n";
    cout << "Sodium: " << f->sodium << " mg\n";
    cout << "Calories: " << f->calories << "\n";
}

void addPatient() {
    Patient p;
    cout << "\nEnter Patient ID: ";
    cin >> p.id;
    cin.ignore();

    cout << "Enter Patient Name: ";
    getline(cin, p.name);

    cout << "Enter Age: ";
    cin >> p.age;
    cin.ignore();

    cout << "Enter Disease (Diabetes/BP/Kidney/etc): ";
    getline(cin, p.disease);
```

```
    cout << "Enter Allergy (or none): ";
    getline(cin, p.allergy);

    patients.push_back(p);
    cout << "Patient added successfully.\n";
}

void checkPatientDiet() {
    int pid, fid;
    cout << "\nEnter Patient ID: ";
    cin >> pid;
    cout << "Enter Food ID: ";
    cin >> fid;

    Patient* p = findPatientById(pid);
    FoodItem* f = findFoodById(fid);

    if (!p || !f) {
        cout << "Patient or Food not found.\n";
        return;
    }

    cout << DietEngine::canPatientEat(*p, *f) << "\n";
}

void addHospital() {
    HospitalContact h;
    cout << "\nEnter Hospital ID: ";
    cin >> h.id;
    cin.ignore();

    cout << "Enter Hospital Name: ";
    getline(cin, h.name);
```

```
    getline(cin, h.phone);

    cout << "Enter Location: ";
    getline(cin, h.location);

    hospitals.push_back(h);
    cout << "Hospital contact added successfully.\n";
}

void viewHospitals() {
    if (hospitals.empty()) {
        cout << "\nNo hospital records found.\n";
        return;
    }

    for (const auto& h : hospitals) {
        cout << "\n-----------------------------\n";
        cout << "ID: " << h.id << "\n";
        cout << "Hospital: " << h.name << "\n";
        cout << "Phone: " << h.phone << "\n";
        cout << "Location: " << h.location << "\n";
    }
}

void groceryQualityCheck() {
    int id;
    cout << "\nEnter Food ID for quality check: ";
    cin >> id;

    FoodItem* f = findFoodById(id);
    if (!f) {
        cout << "Food item not found.\n";
        return;
    }
```

```
        int score = GroceryCheck::qualityScore(*f);
        cout << "\nQuality Score: " << score << "/100\n";
        cout << "Quality Status: " << GroceryCheck::qualityStatus(score) << "\n";
    }

    void saveAll() {
        FileManager::saveFoods(foods);
        FileManager::savePatients(patients);
        FileManager::saveHospitals(hospitals);
        cout << "All data saved successfully.\n";
    }

    void loadAll() {
        FileManager::loadFoods(foods);
        FileManager::loadPatients(patients);
        FileManager::loadHospitals(hospitals);
        cout << "All data loaded successfully.\n";
    }

    void menu() {
        int choice;
        do {
            cout <<
"\n==============================================\n";
            cout << " SMART PERSONAL FOOD SAFETY & HEALTHCARE
ASSISTANT\n";
            cout << "==============================================\n";
            cout << "1. Add Food Item\n";
            cout << "2. View Food Items\n";
            cout << "3. Check Expired Foods\n";
            cout << "4. Check Food Safe To Eat\n";
            cout << "5. Show Nutrition Details\n";
            cout << "6. Add Patient\n";
            cout << "7. Check Patient Can Eat or Not\n";
```

```
        cout << "8. Add Hospital Contact\n";
        cout << "9. View Hospital Contacts\n";
        cout << "10. Grocery Quality Check\n";
        cout << "11. Save All Data\n";
        cout << "12. Load All Data\n";
        cout << "0. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
            case 1: addFood(); break;
            case 2: viewFoods(); break;
            case 3: checkExpiryFoods(); break;
            case 4: checkFoodSafety(); break;
            case 5: showNutrition(); break;
            case 6: addPatient(); break;
            case 7: checkPatientDiet(); break;
            case 8: addHospital(); break;
            case 9: viewHospitals(); break;
            case 10: groceryQualityCheck(); break;
            case 11: saveAll(); break;
            case 12: loadAll(); break;
            case 0: cout << "Exiting program...\n"; break;
            default: cout << "Invalid choice. Try again.\n";
        }
    } while (choice != 0);
  }
};

int main() {
  App app;
  app.menu();
  return 0;
}
```

Output 1: Main Menu

```
==============================================
 SMART PERSONAL FOOD SAFETY & HEALTHCARE ASSISTANT
==============================================
1. Add Food Item
2. View Food Items
3. Check Expired Foods
4. Check Food Safe To Eat
5. Show Nutrition Details
6. Add Patient
7. Check Patient Can Eat or Not
8. Add Hospital Contact
9. View Hospital Contacts
10. Grocery Quality Check
11. Save All Data
12. Load All Data
0. Exit
Enter your choice:
```

Output 2: Add Food Item

```
Enter your choice: 1
Enter Food ID: 101
Enter Food Name: Milk
Enter Category: Dairy
Enter Quantity: 2
Enter Purchase Date (dd mm yyyy): 25 4 2026
Enter Expiry Date (dd mm yyyy): 30 4 2026
Enter Storage Type (Room/Refrigerator/Freezer): Refrigerator
Enter Protein (g): 3.4
Enter Vitamins (example: A,B,C): A,D,B12
Enter Sugar (g): 5
```

```
Enter Sodium (mg): 120
Enter Calories: 150
Bad smell? (1 yes / 0 no): 0
Color changed? (1 yes / 0 no): 0
Leakage? (1 yes / 0 no): 0
Fungus visible? (1 yes / 0 no): 0
Leftover stored days: 1
Food item added successfully.
```

Output 3: View Food Items

```
Enter your choice: 2
-----------------------------
ID: 101
Name: Milk
Category: Dairy
Quantity: 2
Purchase Date: 25/4/2026
Expiry Date: 30/4/2026
Storage: Refrigerator
Protein: 3.4 g
Vitamins: A,D,B12
Sugar: 5 g
Sodium: 120 mg
Calories: 150
```

Output 4: Check Expired Foods

```
Enter your choice: 3
Food: Milk -> NOT EXPIRED
```

Another possible case

```
Food: Bread -> EXPIRED
```

Output 5: Check Food Safe To Eat

```
Enter your choice: 4
Enter Food ID to check safety: 101
SAFE TO EAT
```

Unsafe case 1

```
Enter Food ID to check safety: 102
UNSAFE: Food is expired.
```

Unsafe case 2

```
Enter Food ID to check safety: 103
UNSAFE: Spoilage signs detected.
```

Unsafe case 3

```
Enter Food ID to check safety: 104
UNSAFE: Leftover stored for more than 4 days.
```

Output 6: Show Nutrition Details

```
Enter your choice: 5
Enter Food ID to view nutrition: 101

Nutrition Details for Milk:
Protein: 3.4 g
Vitamins: A,D,B12
Sugar: 5 g
Sodium: 120 mg
Calories: 150
```

Output 7: Add Patient

```
Enter your choice: 6
Enter Patient ID: 201
Enter Patient Name: Ramesh
Enter Age: 52
Enter Disease (Diabetes/BP/Kidney/etc): Diabetes
Enter Allergy (or none): none
Patient added successfully.
```

Output 8: Check Patient Can Eat or Not

Normal case

```
Enter your choice: 7
Enter Patient ID: 201
Enter Food ID: 101
ALLOWED: Patient can eat this food.
```

Diabetes case

```
Enter Patient ID: 201
Enter Food ID: 105
AVOID: High sugar food for diabetic patient.
```

BP case

```
Enter Patient ID: 202
Enter Food ID: 106
AVOID: High sodium food for BP patient.
```

Allergy case

```
Enter Patient ID: 203
```

```
Enter Food ID: 107
NOT ALLOWED: Allergy detected.
```

Output 9: Add Hospital Contact

```
Enter your choice: 8
Enter Hospital ID: 301
Enter Hospital Name: City Care Hospital
Enter Phone: 9876543210
Enter Location: Rajamahendravaram
Hospital contact added successfully.
```

Output 10: View Hospital Contacts

```
Enter your choice: 9
-----------------------------
ID: 301
Hospital: City Care Hospital
Phone: 9876543210
Location: Rajamahendravaram
```

Output 11: Grocery Quality Check

Best quality case

```
Enter your choice: 10
Enter Food ID for quality check: 101
Quality Score: 100/100
Quality Status: Excellent
```

Good quality case

```
Enter Food ID for quality check: 102
Quality Score: 72/100
Quality Status: Good
```

Average quality case

```
Enter Food ID for quality check: 103

Quality Score: 48/100
Quality Status: Average
```

Poor quality case

```
Enter Food ID for quality check: 104

Quality Score: 20/100
Quality Status: Poor
```

Output 12: Save All Data

```
Enter your choice: 11
All data saved successfully.
```

Output 13: Load All Data

```
Enter your choice: 12
All data loaded successfully.
```

Output 14: Exit

```
Enter your choice: 0
Exiting program...
```

Output 15: Full Combined Sample Run

```
==============================================
 SMART PERSONAL FOOD SAFETY & HEALTHCARE ASSISTANT
==============================================
```

```
1. Add Food Item
2. View Food Items
3. Check Expired Foods
4. Check Food Safe To Eat
5. Show Nutrition Details
6. Add Patient
7. Check Patient Can Eat or Not
8. Add Hospital Contact
9. View Hospital Contacts
10. Grocery Quality Check
11. Save All Data
12. Load All Data
0. Exit
Enter your choice: 1

Enter Food ID: 101
Enter Food Name: Milk
Enter Category: Dairy
Enter Quantity: 2
Enter Purchase Date (dd mm yyyy): 25 4 2026
Enter Expiry Date (dd mm yyyy): 30 4 2026
Enter Storage Type (Room/Refrigerator/Freezer): Refrigerator
Enter Protein (g): 3.4
Enter Vitamins (example: A,B,C): A,D,B12
Enter Sugar (g): 5
Enter Sodium (mg): 120
Enter Calories: 150
Bad smell? (1 yes / 0 no): 0
```